Task: Windows ve Linux Installer Altyapısının Geliştirilmesi, CLI Desteği ve Otomatik İmzalama

Amaç

CounterLoad uygulaması için hem Windows hem de Linux işletim sistemlerinde kullanılabilecek bir installer altyapısı geliştirilecektir.

Installer aşağıdaki kullanım biçimlerini desteklemelidir:

- Windows üzerinde grafik arayüzlü kurulum
- Linux masaüstü ortamında grafik arayüzlü kurulum
- Windows ve Linux üzerinde komut satırından sessiz kurulum
- Kullanıcı arayüzü bulunmayan Linux sistemlerinde tamamen komut satırı tabanlı kurulum
- Linux build sunucusunda otomatik installer üretimi
- Cross-compile ile üretilen Windows binary'lerinin installer içine alınması
- Windows binary'lerinin ve Windows installer çıktısının build sunucusundaki `osslsign` uygulamasıyla imzalanması

Mevcut Qt Installer Framework çalışması temel alınacak; ancak Windows ve Linux ihtiyaçlarını daha uygun karşılayan alternatif installer teknolojileri de araştırılacaktır.

Mevcut Durum

Mevcut çalışmada Qt Installer Framework kullanılarak aşağıdaki yapı oluşturulmuştur:

```
QT_counterloadapp/
├── config/
│   └── config.xml
└── packages/
    └── com.counterloadapp.main/
        ├── data/
        └── meta/
            ├── installscript.js
            └── package.xml
```

Installer aşağıdaki komutla üretilmektedir:

```
binarycreator \
    -c config/config.xml \
    -p packages \
    CounterLoadSetup
```

Windows üzerinde çalıştırıldığında `.exe`, Linux üzerinde çalıştırıldığında `.run` installer üretilebildiği belirtilmiştir.

Mevcut yapı kaynak kod deposine taşınmalı, build sunucusunda tekrarlanabilir ve parametreli şekilde çalışacak hale getirilmelidir.

Hedef Mimari

Linux build sunucusunda aşağıdaki akış uygulanmalıdır:

```
Linux uygulamasının build edilmesi
                ↓
Windows uygulamasının cross-compile edilmesi
                ↓
Linux ve Windows staging dizinlerinin hazırlanması
                ↓
Windows EXE ve DLL dosyalarının osslsign ile imzalanması
                ↓
Windows imzalarının doğrulanması
                ↓
Linux installer üretilmesi
                ↓
Windows installer üretilmesi
                ↓
Windows installer dosyasının osslsign ile imzalanması
                ↓
Installer ve imza doğrulama testlerinin çalıştırılması
                ↓
Artifact repository'ye yükleme
```

Örnek çıktı yapısı:

```
artifacts/
├── windows/
│   ├── CounterLoadSetup-1.2.3.exe
│   └── checksums.txt
└── linux/
    ├── CounterLoadSetup-1.2.3.run
    └── checksums.txt
```

Alternatif teknoloji seçilmesi halinde Linux çıktısı `.deb`, `.rpm`, `.AppImage`, `.tar.gz` veya başka uygun bir format olabilir. Nihai format araştırma sonucunda belirlenmelidir.

---

1. Ortak Installer Projesi

Installer projesi mümkün olduğunca Windows ve Linux için ortak kullanılmalıdır.

Önerilen yapı:

```
installer/
├── config/
│   ├── config.xml
│   ├── config-windows.xml
│   └── config-linux.xml
├── packages/
│   └── com.counterloadapp.main/
│       ├── data/
│       └── meta/
│           ├── package.xml
│           └── installscript.js
├── resources/
│   ├── icons/
```

```
│   ├── images/
│   └── licenses/
├── scripts/
│   ├── prepare-package.sh
│   ├── build-installer.sh
│   ├── build-windows-installer.sh
│   ├── build-linux-installer.sh
│   ├── sign-windows-artifacts.sh
│   ├── verify-windows-signatures.sh
│   └── test-installers.sh
└── README.md
```

İşletim sistemine özel ayarlar mümkün olduğunca ayrı konfigürasyon veya scriptlerle yönetilmelidir.

---

2. Windows Installer Gereksinimleri

Windows installer aşağıdaki özellikleri sağlamalıdır:

- Grafik arayüzlü kurulum
- Hedef dizin seçimi
- Başlat menüsü kısayolu oluşturulması
- Opsiyonel masaüstü kısayolu oluşturulması
- Gerekli registry kayıtlarının oluşturulması
- Mevcut kurulumun güncellenmesi
- Uygulamanın kaldırılması
- Sessiz kurulum
- Sessiz kaldırma
- Kurulum loglarının alınabilmesi
- İşlem sonucunun exit code ile döndürülmesi
- Standart kullanıcı ve yönetici yetkisi gerektiren senaryoların yönetilmesi

Örnek kullanım:

```
CounterLoadSetup.exe install `
    --root "C:\Program Files\CounterLoad" `
    --accept-licenses `
    --confirm-command
```

Seçilen teknolojiye göre aşağıdaki gibi standart sessiz kurulum parametreleri de kullanılabilir:

```
CounterLoadSetup.exe /S
```

veya:

```
CounterLoadSetup.exe /VERYSILENT /SUPPRESSMSGBOXES
```

MSI kullanılması halinde:

```
msiexec /i CounterLoadSetup.msi /qn /L*v install.log
```

---

3. Linux Installer Gereksinimleri

Linux installer hem masaüstü hem de sunucu kullanımını desteklemelidir.

Linux masaüstü kurulumu

Grafik masaüstü ortamı mevcutsa installer, Windows installer'a benzer bir kurulum sihirbazı sunmalıdır:

```
Welcome
   ↓
License
   ↓
Installation Directory
   ↓
Component Selection
   ↓
Install
   ↓
Finish
```

Grafik arayüz aşağıdaki ortamlarda test edilmelidir:

- GNOME
- KDE Plasma
- X11
- Wayland

En azından desteklenen ana Linux dağıtımında "Next, Next, Install" şeklinde kullanılabilir bir masaüstü installer sağlanmalıdır.

Qt Installer Framework bu amaçla kullanılacaksa Linux `.run` installer'ın grafik arayüzle çalışması sağlanmalıdır.

Örnek:

```
chmod +x CounterLoadSetup-1.2.3.run
./CounterLoadSetup-1.2.3.run
```

Grafik ortam bulunmayan Linux sistemi

`DISPLAY` veya uygun grafik oturumu bulunmadığında installer hata vererek sonlanmamalıdır.

Aşağıdaki seçeneklerden biri desteklenmelidir:

1. Aynı installer'ın CLI modunda çalıştırılması
2. Ayrı bir komut satırı kurulum scripti sağlanması
3. Dağıtıma özgü `.deb` veya `.rpm` paketlerinin kullanılması

Örnek CLI kurulumu:

```
sudo ./CounterLoadSetup-1.2.3.run \
    install \
    --root /opt/counterload \
    --accept-licenses \
    --confirm-command
```

Alternatif kullanım:

```
sudo ./install.sh \
    --prefix /opt/counterload \
    --non-interactive
```

Linux installer aşağıdaki işlemleri desteklemelidir:

- Grafik arayüzlü kurulum
- CLI üzerinden interaktif kurulum
- CLI üzerinden tamamen sessiz kurulum
- Hedef dizinin parametreyle verilmesi
- Kurulum loglarının alınması
- Exit code döndürülmesi
- Sessiz kaldırma
- Upgrade
- Konfigürasyon dosyalarının korunması
- Desktop entry oluşturulması
- Uygulama ikonunun kurulması
- Gerekliyse systemd service oluşturulması
- Gerekliyse kullanıcı ve grup oluşturulması
- Dosya izinlerinin doğru atanması

Desktop entry gerekiyorsa aşağıdaki konum kullanılmalıdır:

```
/usr/share/applications/
```

İkonlar uygun freedesktop dizinlerine yerleştirilmelidir.

---

4. Build Sunucusu Desteği

Build sunucusu Linux’tur. Installer üretimi bu sunucuda grafik arayüz gerektirmeden gerçekleştirilmelidir.

Ana build komutu aşağıdaki gibi kullanılabilmelidir:

```
./installer/scripts/build-installer.sh \
    --version 1.2.3 \
    --linux-input build/linux/release \
    --windows-input build/windows/release \
```

```
    --output artifacts
```

Script aşağıdaki parametreleri desteklemelidir:

```
--version
--linux-input
--windows-input
--output
--configuration
--build-windows
--build-linux
--sign-windows
--skip-tests
--help
```

Örnek:

```
./installer/scripts/build-installer.sh \
    --version 1.2.3 \
    --linux-input ./dist/linux \
    --windows-input ./dist/windows \
    --output ./artifacts \
    --build-linux \
    --build-windows \
    --sign-windows
```

Build işlemi aşağıdaki şartları sağlamalıdır:

- Kullanıcı etkileşimi gerektirmemelidir.
- Grafik oturuma ihtiyaç duymamalıdır.
- Başarısızlık durumunda sıfırdan farklı exit code dönmelidir.
- Eksik binary veya konfigürasyon bulunursa build durmalıdır.
- İmzalama başarısızsa Windows installer üretilmiş kabul edilmemelidir.
- İmza doğrulanmadan artifact repository'ye yükleme yapılmamalıdır.
- Sürüm numarası tüm installer metadata alanlarında aynı olmalıdır.
- Aynı kaynak girdileriyle tekrarlanabilir çıktı üretilmelidir.

---

5. Windows Cross-Compile Çıktılarının Paketlenmesi

Windows uygulaması Linux build sunucusunda cross-compile edilmektedir.

Installer projesi Windows binary'lerini aşağıdaki gibi ayrı bir staging dizininden almalıdır:

```
staging/
├── windows/
│   ├── CounterLoad.exe
│   ├── *.dll
│   ├── config/
│   └── resources/
└── linux/
    ├── counterload
    ├── lib/
    ├── config/
```

```
    └── resources/
```

Cross-compile çıktıları installer içine alınmadan önce aşağıdaki kontroller yapılmalıdır:

- Ana uygulama executable dosyası mevcut mu?
- Gerekli DLL dosyaları mevcut mu?
- Debug binary yanlışlıkla eklenmiş mi?
- Gerekli runtime kütüphaneleri mevcut mu?
- Binary mimarisi doğru mu?
- Sürüm bilgisi doğru mu?
- Dosyalar başarıyla imzalanmış mı?

Windows mimarisi uygun bir araçla kontrol edilmelidir:

```
file CounterLoad.exe
```

Gerekirse PE header kontrolü için ek araç kullanılmalıdır.

---

6. Windows Kod İmzalama

Windows kod imzalama işlemi Linux build sunucusunda bulunan `osslsign` uygulamasıyla gerçekleştirilecektir.

`signtool.exe` kullanılmayacaktır.

İmzalama scripti build sunucusunda mevcut `osslsign` komutunun gerçek parametrelerine göre hazırlanmalıdır.

Örnek soyut kullanım:

```
osslsign \
    --certificate "${SIGN_CERTIFICATE}" \
    --private-key "${SIGN_PRIVATE_KEY}" \
    --timestamp-url "${TIMESTAMP_URL}" \
    --digest sha256 \
    --input CounterLoad.exe \
    --output CounterLoad.signed.exe
```

Komutun gerçek sözdizimi mevcut build sunucusundaki uygulamanın yardım çıktısından doğrulanmalıdır:

```
osslsign --help
```

veya:

```
osslsign -h
```

İmzalama scripti aşağıdaki gibi çağrılabilmelidir:

```
./installer/scripts/sign-windows-artifacts.sh \
    --input ./staging/windows \
    --certificate "${WINDOWS_SIGN_CERT}" \
```

```
    --timestamp-url "${WINDOWS_TIMESTAMP_URL}"
```

Aşağıdaki dosyalar imzalanmalıdır:

1. Ana Windows uygulama `.exe` dosyası
2. Dağıtılan diğer `.exe` dosyaları
3. İmzalanması gereken `.dll` dosyaları
4. Bootstrapper bulunuyorsa bootstrapper
5. Nihai Windows installer `.exe` dosyası

Doğru sıra:

```
Windows EXE ve DLL dosyalarını imzala
                ↓
İmzaları doğrula
                ↓
İmzalı dosyaları installer içine ekle
                ↓
Windows installer oluştur
                ↓
Nihai installer dosyasını imzala
                ↓
Installer imzasını doğrula
```

İmzalama sırasında aşağıdaki şartlar sağlanmalıdır:

- SHA-256 digest kullanılmalıdır.
- RFC 3161 uyumlu timestamp kullanılmalıdır.
- Sertifika zinciri installer içinde doğru şekilde bulunmalıdır.
- Private key kaynak kod deposine konulmamalıdır.
- Sertifika parolası build loglarına yazılmamalıdır.
- Komut satırı parametreleri secret değerlerini açık olarak göstermemelidir.
- İmzalama başarısızsa pipeline durmalıdır.
- İmzasız Windows artifact yayımlanmamalıdır.

---

7. İmza Doğrulama

İmzalama sonrasında PE dosyalarının Authenticode imzası Linux üzerinde doğrulanmalıdır.

Doğrulama için `osslsign` uygulamasının doğrulama özelliği bulunuyorsa bu kullanılmalıdır.

Örnek soyut kullanım:

```
osslsign verify CounterLoad.exe
osslsign verify CounterLoadSetup-1.2.3.exe
```

Uygulamanın doğrulama desteği bulunmuyorsa aşağıdaki alternatifler araştırılmalıdır:

- `osslsigncode verify`
- `pesign`
- `sbverify` uygunluğu
- OpenSSL tabanlı PE signature doğrulama araçları
- Wine altında Windows doğrulama araçları
- Ayrı Windows test ajanı

Doğrulama sonucunda aşağıdaki bilgiler kontrol edilmelidir:

- İmza mevcut mu?
- Dosya özeti geçerli mi?
- Sertifika zinciri geçerli mi?
- Publisher bilgisi doğru mu?
- Timestamp mevcut mu?
- Timestamp doğrulanabiliyor mu?
- Sertifika süresi sona erse dahi timestamp sayesinde imza geçerli kalıyor mu?

---

8. SmartScreen ve Windows İtibar Testleri

Geçerli bir Authenticode imzası aşağıdaki faydaları sağlar:

- “Unknown publisher” uyarısının engellenmesi
- UAC ekranında yayıncı adının gösterilmesi
- Dosyanın değiştirilmediğinin doğrulanması

Ancak yalnızca sertifikayla imzalama, Microsoft SmartScreen uyarısının hiçbir koşulda gösterilmeyeceğini garanti etmez. SmartScreen ayrıca dosyanın ve yayıncının itibarını da değerlendirebilir.

Aşağıdaki testler yapılmalıdır:

- Windows 10 temiz kurulum
- Windows 11 temiz kurulum
- Microsoft Defender SmartScreen açık sistem
- Edge üzerinden indirme
- Chrome üzerinden indirme
- Mark-of-the-Web içeren dosya
- Yeni sürüm numarasıyla oluşturulmuş installer
- Standart kullanıcı

- Yönetici kullanıcı

Kontrol edilecek sonuçlar:

- Publisher adı doğru mu?
- “Unknown publisher” uyarısı oluşuyor mu?
- SmartScreen uyarısı oluşuyor mu?
- UAC ekranı doğru yayıncıyı gösteriyor mu?
- Dosya imzası geçerli mi?
- Timestamp geçerli mi?

Taskın kabul kriteri “SmartScreen hiçbir zaman uyarı göstermeyecek” olmamalıdır.

Kabul kriteri şu şekilde tanımlanmalıdır:

- Installer güvenilir Authenticode sertifikasıyla imzalanmalıdır.
- “Unknown publisher” uyarısı oluşmamalıdır.
- UAC ekranında doğru kurum adı görüntülenmelidir.
- SmartScreen sonuçları temiz Windows sistemlerde test edilmelidir.
- SmartScreen uyarısı devam ediyorsa sertifika itibarı, dosya itibarı ve dağıtım kanalı açısından nedenleri raporlanmalıdır.

---

9. Linux Paket İmzalama ve Bütünlük

Windows Authenticode sertifikası Linux binary’lerini imzalamak için kullanılmayacaktır.

Linux çıktıları için aşağıdaki seçenekler değerlendirilmelidir:

- SHA-256 checksum
- GPG detached signature
- `.deb` repository imzası
- `.rpm` paket imzası
- AppImage imzası
- Kurum içi artifact repository checksum ve signature mekanizması

Asgari olarak aşağıdaki dosya üretilmelidir:

```
sha256sum CounterLoadSetup-1.2.3.run \
    > CounterLoadSetup-1.2.3.run.sha256
```

GPG altyapısı mevcutsa:

```
gpg --detach-sign \
    --armor \
    CounterLoadSetup-1.2.3.run
```

Linux için kullanılacak nihai imzalama veya bütünlük yöntemi araştırma sonucunda belirlenmelidir.

---

10. Alternatif Installer Teknolojilerinin Araştırılması

Araştırma yalnızca Windows installer araçlarıyla sınırlı olmamalıdır. Aynı Linux build sunucusundan Windows ve Linux installer üretme kabiliyeti temel kriter olmalıdır.

Qt Installer Framework

Araştırılacak konular:

- Linux build sunucusunda çalışması
- Linux `.run` installer üretmesi
- Linux üzerinde grafik arayüz sağlaması
- Headless Linux üzerinde CLI kurulumu
- Windows `.exe` installer üretimi
- Linux üzerinde Windows installer üretme desteği
- Windows installer oluşturmak için Windows binarycreator gerekip gerekmediği
- Wine kullanımı gerekip gerekmediği
- Cross-platform ortak installer tanımı
- Sessiz kurulum
- Update ve maintenance tool
- Uninstall güvenliği
- Kod imzalama sürecine entegrasyon
- Lisans koşulları

Özellikle Linux host üzerinde doğrudan Windows installer üretilebildiği varsayılmamalıdır. Qt Installer Framework araçlarının hedef platform bağımlılığı araştırılmalı ve PoC ile doğrulanmalıdır.

Gerekirse aşağıdaki seçenekler değerlendirilmelidir:

- Linux build sunucusunda Wine ile Windows `binarycreator.exe` çalıştırılması
- Container içinde Windows installer build ortamı
- Ayrı Windows build ajanı
- Windows installer için farklı bir teknoloji kullanılması

CPack

CMake tabanlı proje için aşağıdaki çıktıları üretme kabiliyeti değerlendirilmelidir:

- DEB

- RPM
- TGZ
- NSIS
- WiX
- DragNDrop ve diğer platform paketleri

Avantajları:

- CMake build sistemine doğrudan entegrasyon
- Linux üzerinde `.deb`, `.rpm` ve `.tar.gz` üretimi
- CLI tabanlı build
- CI/CD kullanımına uygunluk

Grafik Linux installer gereksinimini tek başına karşılayıp karşılamadığı ayrıca değerlendirilmelidir.

makeself

Linux için self-extracting `.run` installer üretme seçeneği araştırılmalıdır.

Avantajları:

- Linux üzerinde kolay üretim
- Tamamen komut satırı tabanlı çalışma
- Headless sistem desteği
- Basit CI entegrasyonu

Dezavantajı:

- Yerleşik modern grafik kurulum sihirbazı sağlamaması
- Upgrade ve uninstall özelliklerinin ayrıca geliştirilmesi

DEB ve RPM paketleri

Desteklenen Linux dağıtımları belirliyse native paketler tercih edilebilir.

Araştırılacak başlıklar:

- Debian/Ubuntu için `.deb`
- RHEL/Rocky/Alma/Fedora için `.rpm`
- Paket bağımlılıkları
- Upgrade
- Uninstall
- systemd entegrasyonu
- Menü ve desktop entry entegrasyonu

- Repository üzerinden dağıtım
- Paket imzalama

AppImage

Masaüstü Linux uygulaması için AppImage değerlendirilmelidir.

Araştırılacak başlıklar:

- Kurulum gerektirmeden çalıştırma
- Desktop entegrasyonu
- İmzalama
- Güncelleme desteği
- Sistem bağımlılıkları
- Sunucu ve headless kurulumlarına uygunluk

Inno Setup veya NSIS

Windows installer için Linux build sunucusunda Wine altında çalıştırılabilirlik araştırılmalıdır.

Değerlendirilecek başlıklar:

- Cross-platform build ortamı
- Wine altında compiler çalıştırma
- Sessiz kurulum
- Authenticode imzalama
- Upgrade ve uninstall
- Qt Installer Framework'e göre bakım maliyeti

WiX Toolset

MSI gereksinimi varsa değerlendirilmelidir.

Ancak Linux build sunucusunda çalışma ve Windows dışından MSI üretme yeteneği PoC ile doğrulanmalıdır.

---

11. Önerilen Teknik Yaklaşım

İlk PoC aşağıdaki yaklaşımla yapılmalıdır:

Birinci tercih

Qt Installer Framework ile:

- Linux `.run` GUI installer

- Linux `.run` CLI installer
- Windows `.exe` GUI installer
- Windows `.exe` CLI installer
- Ortak paket tanımı
- Linux build sunucusunda otomatik build

Aşağıdaki durumlar başarıyla doğrulanırsa Qt Installer Framework ile devam edilebilir:

- Linux host üzerinde Windows installer güvenilir şekilde üretilebiliyorsa
- Wine kullanımı stabil çalışıyorsa
- CLI ve headless Linux kurulumu ihtiyaçları karşılanıyorsa
- Uninstall işlemi güvenli şekilde yönetilebiliyorsa

Alternatif hibrit yaklaşım

Qt Installer Framework Windows cross-build konusunda sorun çıkarırsa:

```
Windows:
    Inno Setup veya NSIS
    +
    osslsign ile Authenticode imzalama

Linux masaüstü:
    Qt Installer Framework .run
    veya AppImage

Linux sunucu:
    DEB/RPM
    veya makeself tabanlı CLI installer
```

Hibrit yapıda ortak sürüm, dosya listesi ve staging altyapısı korunmalıdır.

---

12. Test Senaryoları

Windows

- Grafik arayüzle temiz kurulum
- CLI ile sessiz kurulum
- Varsayılan dizine kurulum
- Özel dizine kurulum
- Upgrade
- Sessiz kaldırma
- GUI üzerinden kaldırma
- İmza doğrulama

- SmartScreen testi
- UAC publisher kontrolü
- Eksik bağımlılık testi
- Bozuk installer testi

Linux masaüstü

- GNOME üzerinde GUI kurulum
- KDE üzerinde GUI kurulum
- X11 üzerinde kurulum
- Wayland üzerinde kurulum
- Menü kısayolu kontrolü
- Desktop icon kontrolü
- Upgrade
- GUI kaldırma
- CLI kaldırma

Linux headless

- `DISPLAY` olmadan kurulum
- Sessiz kurulum
- Özel prefix ile kurulum
- Root yetkisi olmadan kullanıcı dizinine kurulum
- Root ile `/opt` altına kurulum
- Sessiz kaldırma
- Exit code kontrolü
- Log dosyası kontrolü

Build sunucusu

- Windows ve Linux çıktılarının tek pipeline’da üretilmesi
- Yalnızca Windows çıktısının üretilmesi
- Yalnızca Linux çıktısının üretilmesi
- Sertifika erişilemezken build’in durması
- İmzalama başarısızken artifact yayımlanmaması
- Eksik cross-compile çıktısında build’in durması
- Aynı sürümün tekrar üretilmesi

- Artifact isimlerinin doğru olması

Teslimatlar

1. Windows installer projesi
2. Linux installer projesi
3. Ortak staging ve paket hazırlama altyapısı
4. Windows grafik installer çıktısı
5. Windows sessiz kurulum desteği
6. Linux grafik installer çıktısı
7. Linux CLI ve headless kurulum desteği
8. Linux build sunucusunda çalışan build scriptleri
9. Cross-compile Windows çıktılarının paketlenmesi
10. `osslsign` tabanlı Windows imzalama scripti
11. Windows imza doğrulama scripti
12. Linux checksum veya imzalama altyapısı
13. CI/CD pipeline entegrasyonu
14. SmartScreen test raporu
15. Installer alternatifleri karar matrisi
16. Kurulum, upgrade ve kaldırma testleri
17. Geliştirici ve operasyon kullanım dokümanı
18. Sertifika ve sürüm güncelleme prosedürü

Kabul Kriterleri

- Linux build sunucusunda kullanıcı etkileşimi olmadan installer üretilebilmelidir.
- Windows cross-compile çıktıları doğru şekilde Windows installer içine alınmalıdır.
- Windows executable ve DLL dosyaları installer oluşturulmadan önce `osslsign` ile imzalanmalıdır.
- Nihai Windows installer `osslsign` ile imzalanmalıdır.
- İmzalar pipeline içinde doğrulanmalıdır.
- İmzasız veya imzası geçersiz Windows artifact yayımlanmamalıdır.
- Windows installer grafik arayüzden kurulabilmelidir.

- Windows installer sessiz modda kurulup kaldırılabilmelidir.
- Linux masaüstü ortamında grafik kurulum sihirbazı çalışmalıdır.
- Grafik ortam bulunmayan Linux sisteminde CLI kurulumu çalışmalıdır.
- Linux installer sessiz modda kurulup kaldırılabilmelidir.
- Windows ve Linux installer'lar anlamlı exit code döndürmelidir.
- Installer logları alınabilmelidir.
- Upgrade sırasında kullanıcı konfigürasyonu korunmalıdır.
- Uninstall yalnızca installer tarafından yönetilen dosyaları kaldırmalıdır.
- Linux desktop entry ve ikon entegrasyonu doğrulanmalıdır.
- Windows UAC ekranında doğru publisher adı gösterilmelidir.
- "Unknown publisher" uyarısı oluşmamalıdır.
- SmartScreen sonuçları temiz Windows ortamında raporlanmalıdır.
- Installer teknolojisi PoC ve karar matrisi sonucunda seçilmelidir.

Tahmini Süre

Çalışma	Süre
Mevcut Qt Installer Framework yapısının incelenmesi	0,5 gün
Qt IFW ile Windows ve Linux cross-platform PoC	1,5 gün
Alternatif installer teknolojilerinin araştırılması	1 gün
Ortak staging ve paket hazırlama altyapısı	1 gün
Windows GUI ve CLI installer geliştirmesi	1,5 gün
Linux GUI ve headless CLI installer geliştirmesi	2 gün
`osslsign` ile Windows imzalama ve doğrulama	1 gün
Linux checksum veya paket imzalama altyapısı	0,5 gün
Build sunucusu ve CI/CD entegrasyonu	1,5 gün
Windows ve Linux kurulum testleri	1,5 gün
SmartScreen ve temiz Windows testleri	0,5 gün
Dokümantasyon	0,5 gün
Toplam	**13 iş günü**

Önerilen planlama

Task için başlangıç tahmini:

13 adam-gün

Çalışma iki aşamaya bölünebilir:

Aşama 1 — Araştırma ve PoC

3 iş günü

- Qt Installer Framework cross-platform build araştırması
- Linux build sunucusunda Windows installer üretme testi
- Linux GUI ve headless CLI testi
- Alternatiflerin karar matrisi
- `osslsign` kullanımının doğrulanması

Aşama 2 — Ürünleştirme

10 iş günü

- Installer geliştirmesi
- İmzalama
- Pipeline entegrasyonu
- Windows ve Linux testleri
- Dokümantasyon

PoC sonunda Qt Installer Framework'ün Linux host üzerinde Windows installer üretimi için uygun olmadığı görülürse hibrit installer mimarisi seçilecektir. Bu durumda toplam süre kullanılan Windows installer teknolojisine göre **2–3 iş günü artabilir**.